ChatGPT
Sora
Explore GPTs
Library

Yesterday
Holy File Footage Query
MongoDB vs SQL Mapping

Previous 7 Days
أسئلة في الفيزياء
الملاحظة والتجربة في العلم
Binary Subtraction and Borrow
Express Static Explanation

Previous 30 Days
Favicon.ico Request Handling
Physics Argument Clarification
ضعف دافعية الطلاب
Redox-flow Vanadium Battery
Package-lock.json Explanation
Execute CMD Script

why doesn't this code work
let a = ["asss-asss", "fda-fad"];
a.forEach(e => e.split('-'))
console.log(a)
I want to put the split of each string in its old place in the array a

Good question!
Your code:

```javascript
let a = ["asss-asss", "fda-fad"];
a.forEach(e => e.split('-'));
console.log(a);
```

doesn't work because `.forEach` just *loops* over the array — but it doesn't **replace** the elements.
You're calling `e.split('-')` inside `forEach`, but you're **not assigning** the result back into the array.

You probably want to do something like this:

```javascript
let a = ["asss-asss", "fda-fad"];
a = a.map(e => e.split('-'));
console.log(a);
```

✅ Now it will output:

```javascript
[ [ 'asss', 'asss' ], [ 'fda', 'fad' ] ]
```

Explanation:

- `.map` creates a **new array** with the **results** of the function you apply to each element.
- `e.split('-')` splits each string into an array of parts.

If you *really* want to use `.forEach` for some reason, you could manually assign:

```javascript
let a = ["asss-asss", "fda-fad"];
a.forEach((e, i) => a[i] = e.split('-'));
console.log(a);
```

But honestly, `map` is cleaner here.

Would you like me to show you a visual "before and after" of what happens too? 🚀

what happens in this code
let b = ">=";
console.log({[b]: true})
how an array can be an object property

Alright, let's walk through your code:

```javascript
let b = ">=";
console.log({ [b]: true });
```

What's happening here is:

- You are **creating an object**, and inside the `{}` you are using **computed property names**.
- `[b]` means: *take the value of the variable* `b` (which is the string `">="`) and **use it as the key** of the object.

✅ So this code **creates** the object:

```javascript
{ ">=": true }
```

Key point:
When you write `[b]`, `b` **gets evaluated**, and its **value** becomes the key.

> **Now about your second question:**
> **How can an array be an object property?**

In JavaScript, **object keys are always either**:

- strings (`"something"`) or

- symbols.

When you use something else (like an **array**) as a key, JavaScript **automatically converts** it into a string.

Example:

```javascript
let arr = [1,2];
let obj = {
    [arr]: "hello"
};
console.log(obj);
```

👀 This prints:

```javascript
{ "1,2": "hello" }
```

⚡ Why? Because:

- `arr` is `[1,2]`
- When you use it as a key, JavaScript does `arr.toString()` → `"1,2"`
- So the actual object becomes `{ "1,2": "hello" }`

Summary:

- When you do `{ [something]: value }`, `something` is **evaluated** and **converted to a string** if needed.
- Arrays become keys by getting **stringified**.
- Object keys are **always strings or symbols** (unless you're using a `Map`, which can have *any* object as a key).

Would you like me to show a crazy example with nested arrays as keys too? 🤯 It's kind of fun!

شرح البوابات المنطقية

Upgrade plan
More access to the best models

Ask anything

Search | Reason | Deep research | Create image

ChatGPT can make mistakes. Check important info.